Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

Simulación de Desplazamiento de Drones en Paralelo

202404224 – Alejandro Emmanuel Alvarez Velásquez

Resumen

El presente informe detalla el desarrollo y funcionamiento de un sistema informático diseñado en el lenguaje C# para la simulación y optimización del despliegue coordinado de una flota de drones. El objetivo principal del proyecto es procesar configuraciones estructuradas en archivos XML para determinar la secuencia de movimientos óptima que permita a un conjunto de drones emitir mensajes luminosos en un tiempo mínimo. Para lograr una gestión eficiente de la información, se implementaron Tipos de Datos Abstractos (TDA), específicamente listas simples enlazadas, garantizando un manejo dinámico de la memoria. Asimismo, el sistema integra la herramienta Graphviz para la generación de representaciones gráficas de los sistemas de drones y las rutas de simulación, facilitando el análisis visual de las estructuras de datos y los resultados obtenidos.

Palabras clave

1. Programación Orientada a objetos
2. Listas Enlazadas
3. Simulación en Paralelo
4. XML
5. Graphviz

Abstract

This report details the development and operation of a computer system designed in the C# language for the simulation and optimization of the coordinated deployment of a drone fleet. The main objective of the project is to process structured configurations in XML files to determine the optimal movement sequence that allows a set of drones to emit light messages in minimum time. To achieve efficient information management, Abstract Data Types (ADT) were implemented, specifically singly linked lists, ensuring dynamic memory management. Additionally, the system integrates the Graphviz tool for generating graphical representations of drone systems and simulation routes, facilitating the visual analysis of data structures and the results obtained.

Keywords

1. *Objet Oriented Programming*
2. *Linked Lists*
3. *Parallel Simulation*
4. *XML*
5. *Graphviz*

Introducción

En el ámbito de la ingeniería de software, la Programación Orientada a Objetos (POO) se presenta como un paradigma esencial para modelar sistemas complejos mediante la abstracción y la organización de datos en entidades lógicas. Este enfoque permite la creación de Tipos de Datos Abstractos (TDA), cuya implementación oculta la complejidad interna y facilita la interacción con componentes externos. En proyectos de simulación, como el desplazamiento coordinado de flotas de drones, la capacidad de abstraer comportamientos y atributos es vital para garantizar la escalabilidad del sistema.El presente proyecto aborda la sinergia entre la POO, el uso de archivos XML como medio de transferencia de datos estructurados y el empleo de estructuras de datos lineales para la gestión de información en tiempo real. Específicamente, se utiliza el lenguaje C# para desarrollar una solución que procesa configuraciones de sistemas de drones, calcula rutas óptimas de movimiento en paralelo y visualiza los resultados mediante la herramienta Graphviz. A través de esta implementación, se demuestra cómo la correcta selección de estructuras de datos y algoritmos de optimización puede resolver problemas logísticos de alta complejidad de manera eficiente

1. Programación Orientada a Objetos:

La programación orientada a objetos representa un paradigma de programación en el cual la estructuración del software se fundamenta en torno a objetos o datos, en contraposición al uso de funciones y lógica. Su enfoque radica en los objetos que los programadores requieren manipular, en vez de centrarse en la lógica necesaria para llevar a cabo dicha manipulación. Un objeto se conceptualiza como un conjunto de atributos y comportamientos distintivos. (Universidad Europea, 2022) Por lo tanto, la característica central de este enfoque reside en su capacidad para manejar objetos, cada uno de los cuales posee un tipo o clase específica. Estas clases pueden heredar atributos de clases superiores, o superclases. Esta particularidad impulsa su implementación en proyectos extensos y complejos que necesitan actualizaciones periódicas.

2. Clase:

Una clase en la programación orientada a objetos funciona como una plantilla que establece las propiedades y acciones de una entidad. Se puede visualizar como un patrón a partir del cual es posible crear instancias con características y comportamientos similares. Básicamente, una clase define tanto los atributos como los comportamientos que caracterizan a una entidad.

3. Abstracción:

La abstracción es cuando el usuario interactúa solo con los atributos y métodos seleccionados de un objeto, utilizando herramientas simplificadas de alto nivel para acceder a un objeto complejo. En la programación orientada a objetos, los programas suelen ser muy grandes y los objetos se comunican mucho entre sí. El concepto de abstracción facilita el mantenimiento de un código de gran tamaño, donde a lo largo del tiempo pueden surgir diferentes cambios. Así, la abstracción se basa en usar cosas simples para representar la complejidad. Los objetos y las clases representan código subyacente, ocultando los detalles complejos al usuario. Por consiguiente, supone una extensión de la encapsulación. Siguiendo

con el ejemplo del coche, no es necesario que conozcas todos los detalles de cómo funciona el motor para poder conducirlo. (Martínez, 2020)

4. **Graphviz:**

Graphviz constituye un software de código abierto de distribución gratuita orientado a la creación de representaciones gráficas, las cuales permiten visualizar datos estructurales en forma de diagramas. Su aplicabilidad es amplia y abarca campos tan diversos como el análisis de redes, la bioinformática, la ingeniería de software, las bases de datos, el diseño de páginas web y el aprendizaje automático. Además, proporciona interfaces gráficas diseñadas para diferentes dominios. (Takeyas, 2012) El conjunto de herramientas Graphviz engloba una diversidad de programas diseñados para la creación de diagramas. Entre estas opciones, el lenguaje "dot" sobresale como la herramienta principal destinada a elaborar representaciones de estructuras jerárquicas o de capas dirigidas. Este lenguaje puede ser ejecutado como un programa desde la línea de comandos, a través de un servicio de visualización web o mediante una interfaz gráfica compatible. (Takeyas, 2012)

5. **Tipos de Datos Abstractos:**

Es un modelo que define valores y las operaciones que se pueden realizan sobre ellos. Y se denomina abstracto ya que la intención es que quien lo utiliza, no necesita conocer los detalles de la representación interna o bien el cómo están implementadas las operaciones. Es por esto una práctica que nos provee un grado de abstracción que permite desacoplar al código que usa un TDA de aquel código que lo implementa.

6. **Estructura de Datos:**

Las estructuras de datos son aquellas que nos permiten, como desarrolladores, organizar la información de manera eficiente, y en definitiva diseñar la solución correcta para un determinado problema. Una estructura de datos nos permite trabajar en un algo nivel de abstracción almacenando información para luego acceder a ella, modificarla y manipularla.

7. **Estructura de Datos Lineales:**

Las estructuras de datos lineales son aquellas en las que los elementos ocupan lugares sucesivos en la estructura y cada uno de ellos tiene un único sucesor y un único predecesor, es decir, sus elementos están ubicados uno al lado del otro relacionados en forma lineal. (SoyHenry, 2022) Las estructuras lineales de datos se caracterizan porque sus elementos están en secuencia, relacionados en forma lineal, uno luego del otro. Cada elemento de la estructura puede estar conformado por uno o varias subelementos que pueden pertenecer a cualquier tipo de dato.

Existen tres tipos de estructuras de datos lineales:

- Listas Enlazadas
- Pilas
- Colas

8. **Listas Enlazadas Simples:**

La estructura de datos conocida como lista enlazada se presenta como un Tipo de Dato Abstracto que proporciona un método organizado de almacenamiento de información. En este sentido, guarda similitudes con los vectores, aunque se diferencia en su carácter dinámico. A diferencia de

los vectores, la lista enlazada no requiere conocimiento previo de la cantidad de elementos a contener. En el contexto de una lista enlazada simple, cada elemento señala al siguiente, a excepción del último que carece de sucesor y, en consecuencia, su enlace adopta el valor nulo. Los elementos adoptan la forma de registros que incorporan tanto los datos a almacenar como un enlace que conecta con el siguiente componente. En ocasiones, se les atribuye a estos elementos el nombre de "nodos" dentro de la lista. En las estructuras de datos, las listas enlazadas se construyen con elementos que están ubicados en una secuencia. Aquí, cada elemento se conecta con el siguiente a través de un enlace que contiene la posición del siguiente elemento. De este modo, teniendo la referencia del principio de la lista podemos acceder a todos los elementos de la misma (SoyHenry, 2022)

9. **Archivos XML:**

XML consiste en un lenguaje de marcado creado por el W3C con la finalidad de definir una sintaxis para la codificación de documentos, que tanto los usuarios como las propias máquinas en sí puedan ser capaces de leer. Para ello, lo hace mediante la utilización de una serie de etiquetas que definen la estructura que posee el documento en cuestión, además de cómo debe ser transportado y almacenado. (Pérez, 2020) Podemos establecer una comparación con otro lenguaje de marcado con el cual es probable que estemos más familiarizados: el lenguaje de marcado de hipertexto (HTML), ampliamente empleado en la codificación de páginas web. HTML emplea una serie de símbolos de marcado predefinidos que delinean el formato del contenido de una página web. No obstante, una distinción fundamental entre XML y HTML radica en su extensibilidad. Mientras HTML opera con un conjunto predeterminado de símbolos de marcado, XML se distingue por su naturaleza extensible.

Conclusiones

1. El desarrollo de este sistema en el lenguaje C# permitió demostrar la eficacia de la Programación Orientada a Objetos para gestionar simulaciones complejas en paralelo. Mediante la abstracción de entidades como drones, sistemas y mensajes, se logró una arquitectura modular que facilita el procesamiento de múltiples instrucciones de vuelo de manera simultánea, optimizando el tiempo de respuesta del sistema.

2. La implementación de Tipos de Datos Abstractos, específicamente la lista simple enlazada, resultó fundamental para garantizar un manejo dinámico y eficiente de la memoria. Al no depender de colecciones nativas, el sistema demuestra un control total sobre la estructura de los datos, permitiendo que la flota de drones y las configuraciones de altura crezcan según las necesidades del archivo XML sin comprometer el rendimiento del software.

3. La integración de herramientas de visualización como Graphviz aporta un valor crítico al proyecto, ya que permite transformar estructuras de datos abstractas en representaciones gráficas comprensibles. Esta capacidad no solo facilita la validación técnica de las listas enlazadas y los nodos, sino que también permite al usuario final analizar visualmente la configuración de los sistemas y el flujo de los mensajes procesados, cumpliendo así con los estándares de documentación técnica profesional.

Referencias bibliográficas

C. J. Date, (1991). An introduction to Database Systems. Addison-Wesley Publishing Company, Inc.

https://digitalgrow.es/que-es-una-clase-en programacion/
https://keepcoding.io/blog/que-es-la-abstraccion-en programacion/

Deitel, P. J., & Deitel, H. M. (2017). C# 6 for Programmers. Prentice Hall.

Knuth, D. E. (1997). The Art of Computer Programming, Volume 1: Fundamental Algorithms. Addison-Wesley Professional.

Anexos

1. **Tabla de Actividades**

Semana	Actividad
Semana 1	Creación de análisis de requerimientos y diseño de la arquitectura de clases (POO). Creación de las estructuras de datos base.
Semana 2	Implementación del LectorXML utilizando XmlDocument. Desarrollo de la lógica de almacenamiento para Sistemas de Drones y Drones configurados.
Semana 3	Desarrollo del algoritmo de simulación paso a paso para el cálculo del tiempo óptimo. Implementación del controlador y las vistas en ASP.NET Core (MVC).
Semana 4	Integración con la API de Graphviz para la generación de gráficas de sistemas y mensajes. Pruebas finales, corrección de errores en la interfaz y redacción de documentación.

Tabla 1. Cronograma de Actividades
Fuente: Elaboración Propia

2. Diagrama de Clases

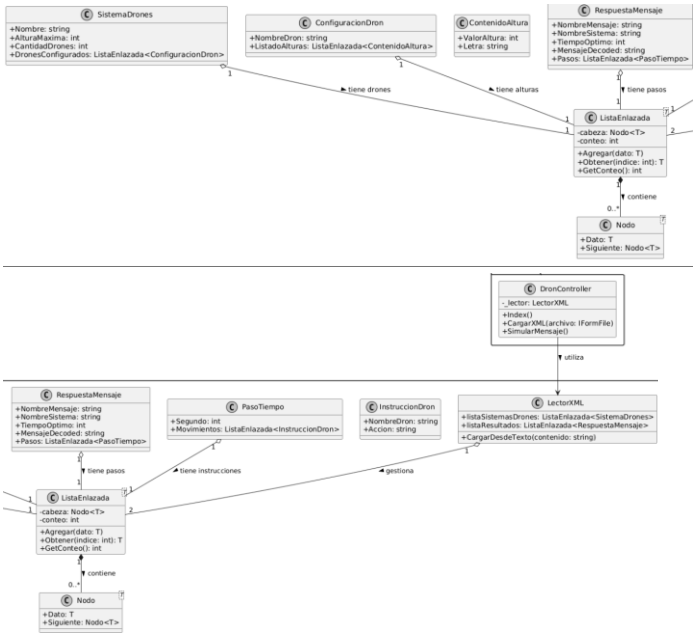

Figura 1. Diagrama de Clases
Fuente: Elaboración Propia

3. Diagrama de Listas

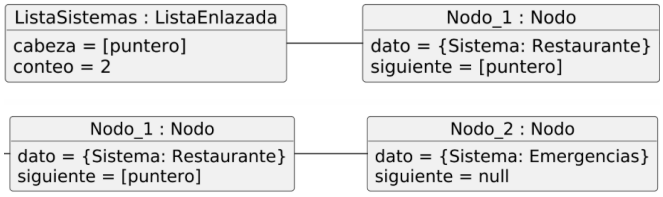

Figura 2. Diagrama de Listas
Fuente: Elaboración Propia